



Birla Institute of Technology & Science, Pilani

Work-Integrated Learning Programmes Division

MTech in Artificial Intelligence and Machine Learning

S2_2025-2026, AIMLCZG557/AECLZG557

Assignment 1 – PS3 - Rescue Robot - [Weightage 12%]

Read through this entire document very carefully before you start!

After a major earthquake, an autonomous rescue robot is deployed to locate trapped survivors inside partially collapsed buildings. The robot must avoid unstable structures, toxic gas zones, and blocked passages while using GBFS to quickly reach survivors.

Problem Statement

Design an intelligent agent using Greedy Best-First Search (GBFS) to navigate from the Start node (S) to the Goal node while avoiding obstacles.

Agent rules, goals and directions

- The agent is dropped at the earthquake location.
- The environment is of 8X8 grid, and the agent is tasked to find the survivors and rescue them.
- In the given environment, the agent scanned there are 3 survivors at different locations.
- The agent should rescue the closest survivor from the agent location based on the heuristic estimate. The minimum is the heuristic estimate from the agent location to the survivor node. When using the Heuristic 1, then it is the minimum Manhattan distance, and when using the Heuristic 2 (risk-aware heuristic) then it is the minimum risk-aware heuristic.
- Once the agent reaches a survivor it should rescue and mark the node as the safe passage node. So the agent can pass through without any penalties. The agent can start and reach other survivors.
- The agent should select the next survivor. In our environment there are three survivors. The agent should re-evaluate heuristics from the current position to the

remaining survivor nodes. Repeat this until all the survivor nodes are reached or when the survivor is not reachable.

- The agent can move orthogonally (Up, Down, Left, Right) from any given location.
- Ex: From (2,2) the possible moves are: (2,0), (2,2), (1,1) (3,1)
- **Diagonal Movement:** Not allowed anywhere in the execution.
- **Cost:**
 - Safe passage (.) = 1 unit
 - Toxic zone (T) = 1 unit (but affects heuristic)
 - Blocked (#) = infinite (impassable)
- Though the agent can take risk and move through the toxic path, it should minimize traversing through toxic nodes.

Environment

	0	1	2	3	4	5	6	7
0	S	.	.	#	.	T	.	.
1	#	.	T	#	.	.	.	.
2	.	.	.	.	T	.	#	.
3	.	#	#	.	.	.	.	.
4	T	.	.	.	#	#	.	P
5	.	.	.	T	.	.	.	.
6	.	#	.	.	P	#	T	.
7	.	.	.	T	.	P	.	.

Legend

- **S** - Start Position (0,0): Robot deployment point
- **P** - Goal/Survivor: Person trapped at coordinates (4,7), (6,4), (7,5)
- **#** - Blocked Passages: Impassable debris/collapsed structures
- **T** - Toxic Gas Zones: Traversable but risky (adds penalty)

- . - Safe Passages: Normal traversable cells

Agent Directions/Movements

- The agent can move orthogonally (Up, Down, Left, Right) from any given location.
- Ex: From (2,2) the possible moves are: (2,0), (2,2), (1,1) (3,1)
- **Diagonal Movement:** Not allowed anywhere in the execution.
- **Cost:**
 - Safe passage (.) = 1 unit
 - Toxic zone (T) = 1 unit (but affects heuristic)
 - Blocked (#) = infinite (impassable)
- Though the agent can take risk and move through the toxic path, it should minimize traversing through toxic nodes.

Heuristics:

Solve the agent problem using the below heuristics

Heuristic 1: Manhattan Distance

$$h_1(n) = |x_{\text{goal}} - x_{\text{current}}| + |y_{\text{goal}} - y_{\text{current}}|$$

- Pure distance-based estimation
- Ignores environmental hazards
- Admissible but not necessarily safe

Heuristic 2: Risk-Aware Heuristic

$$h_2(n) = h_1(n) + \alpha T(n)$$

- $h_1(n)$ = Manhattan Distance
- $T(n)$ = Number of toxic zones near the current node orthogonally (up, down, right, left)
 - $T(n) \in \{0,1,2,3,4\}$, such that $T(n)$ is equal to the number of orthogonal toxic zones
- α = Risk penalty weight. $\alpha=2$.

DELIVERABLES:

1. Explain the PEAS for the problem.
2. Implement the given problem using GBFS with the appropriate data structure
3. The program should
 - a. Have the executable code without any issues
 - b. Implement both the heuristics

- c. Construct initial grid and visual representation of the same. You can choose the appropriate icons for agent, toxic zones and non-passable zones.
 - d. Frontier nodes at each iteration
 - e. Explored nodes
 - f. Heuristic values
 - g. Selected next node
 - h. Final path to each survivor
 - i. Toxic zones crossed
 - j. Rescue sequence
 - k. Number of tries to reach each survivor.
4. Complexity Analysis: Visual representation is preferred where possible.
 - a. provide the nodes explored,
 - b. time complexity in the Big O notation
 - c. execution time
 - d. memory usage
5. Heuristic function Analysis
 - a. Describe which heuristic is better in terms of reaching the goal.
 - b. Visual representation of the heuristic values to reach the survivors.
 - c. Visual representation of the heuristic values vs time to reach the survivors.
6. Identify situations where the agent becomes trapped or explores inefficient regions. Show the instances in a tabular format containing heuristic (h_1 or h_2), heuristic value, trapped at node, next iteration to come out of the trap.
 - a. NOTE: A trap could be a dead-end or infinite loop.
7. Implement and compare the problem with A* using heuristic 1.
 - a. Compare which algorithm is complete
 - b. Compare the nodes expanded using GBFS and A* with heuristic 1.
 - c. Compare which algorithm is fast enough to rescue all the survivors.
 - d. Provide the comparison in a table format or a chart.
8. Documentation
 - a. Document the program clearly.
 - b. Provide the execution flow or sequence diagram
9. Output of the program should be clear
 - a. Visual representation is required for each survivor.
 - b. Update the initial grid and visually represent the updated grid for the next survivor.
10. Complexity Analysis: Visual representation is preferred where possible.
 - a. provide the nodes explored,
 - b. time complexity in the Big O notation
 - c. execution time

d. memory usage

11. Conclusion:

- a. Provide your analysis on the GBFS algorithm in terms of the optimality and completeness.

Deliverables:

- PDF document **designPSXX_<group id>.pdf** detailing your solution.
- **[Group id] _Contribution.xlsx** mentioning the contribution of each student in terms of percentage of work done. Columns must be “Student Registration Number”, “Name”, “Percentage of contribution out of 100%”. If a student did not contribute at all, it will be 0%, if all contributed then 100% for all.
- **inputPSXX.txt** file used for testing
- **outputPSXX.txt** file generated while testing
- .py file containing the python code. Create a single notebook. Do not fragment your code into multiple files
- Zip all of the above files including the design document in a folder with the name:
 - **[Group id] _A1_PSXX_XXXXXXXXXX.zip** and submit the zipped file.
 - Group Id should be given as **Gxxx** where **xxx** is your group number. For example, if your group is 26, then you will enter **G026** as your group id.

Instructions:

- It is compulsory to make use of the data structure(s) / algorithms mentioned in the problem statement.
- Ensure that all data structure insert and delete operations throw appropriate messages when their capacity is empty or full. Also ensure basic error handling is implemented.
- For the purposes of testing, you may implement some functions to print the data structures or other test data. But all such functions must be commented before submission.
- Make sure that you read, understand, and follow all the instructions
- Ensure that the input, prompt and output file guidelines are adhered to. Deviations from the mentioned formats will not be entertained.
- The input, prompt and output samples shown here are only a representation of the syntax to be used. Actual files used to evaluate the submissions will be different. Hence, do not hard code any values into the code.

- Please note that the design document must include:
 - One alternate way of modeling the problem with the performance implications.
 - Writing a good technical report and well documented code is an art. Your report cannot exceed 4 pages. Your code must be modular and quite well documented.
 - You may ask queries in [this dedicated discussion section](#). Beware that only hints will be provided and queries asked **in other channels like MS Teams, emails, Taxila inbox will not be responded to.**

Deadline

1. The strict deadline for submission of the assignment is **8th June 2026 11:55 PM IST (Monday)**.
2. The deadline has been set considering extra days from the regular duration in order to accommodate any challenges you might face. No further extensions will be entertained.
3. Late submissions will be evaluated as below. Submissions made between:
 - a. 8th June 2026 11:56PM to 9th June 2026 11:55PM → Deduct 2M for late submission and evaluation for 10 marks.
 - b. 9th June 2026 11:56PM to 10th June 2026 11:55PM → Deduct 6M for late submission and evaluation for 6marks.
 - c. Beyond 10th June 2026 11:55PM → No evaluations, 0 marks will be awarded.

How to submit

- This is a group assignment.
- Each group has to make one submission (only one, no resubmission) of solutions.
- Each group should zip all the deliverables in one zip file and name the zipped file as [Group id] _A1_PSXX_XXXXXXXXXX.zip and submit the zipped file.
- Group Id should be given as Gxxx where xxx is your group number. For example, if your group is 26, then you will enter G026 as your group id.
- Assignments should be submitted via Taxila > Assignment section. Assignments submitted via other means like email etc. will not be graded.

Evaluation

- The assignment carries **12 Marks**.
- Grading will depend on:
- Fully executable code with all functionality working as expected
 - Well-structured and commented code
 - Accuracy of the design document.
 - Every bug in the functionality will have negative marking.
 - Marks will be deducted if your program fails to read the input file used for evaluation due to change / deviation from the required syntax.
- We encourage students to take the upcoming assignments and examinations seriously and submit only original work. Please note that plagiarism in assignments will be taken seriously. Refer to the detailed policy [here](#).
- Source code files which contain compilation errors will get at most 25% of the value of that question.

Readings

Artificial Intelligence: A Modern Approach Textbook by Peter Norvig and Stuart J. Russell, 4th Edition.